

REPUBLIQUE DU SENEGAL



Un peuple _ Un but _ Une foi

Ministère de l'Enseignement Supérieur,
de la Recherche et de l'Innovation

Université Assane Seck de Ziguinchor
Faculté des Sciences et Technologies
Département d'Informatique

Mémoire de Master 1 – Réseaux Avancés

Thème :

Architecture SDN pour la Gestion Dynamique du Trafic et la Sécurité Réseau

Auteure :
Aminata Thior

Encadrant :
Pr. Malick Ndoeye

Année universitaire 2024–2025

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

En premier lieu, je remercie sincèrement mon encadrant, **Professeur Malick Ndoye**, pour sa disponibilité, ses conseils avisés et son soutien tout au long de ce travail. Sa rigueur scientifique et ses orientations ont été déterminantes dans la réussite de ce projet.

Je remercie également l'ensemble du corps enseignant du **Département d'Informatique** de l'Université Assane Seck de Ziguinchor pour la qualité de la formation dispensée.

Je tiens à exprimer ma reconnaissance à ma **famille**, pour leur soutien indéfectible, leur patience et leurs encouragements constants durant toutes mes années d'études.

Enfin, je remercie mes camarades de promotion pour les échanges enrichissants et l'entraide tout au long de cette année.

Résumé

Les réseaux traditionnels souffrent de rigidité et d'un manque de visibilité globale, rendant difficile la gestion dynamique du trafic et la réponse rapide aux menaces de sécurité. Le paradigme SDN propose une approche innovante en séparant le plan de contrôle du plan de données.

Ce projet présente la conception et le déploiement d'une architecture SDN complète basée sur le contrôleur Ryu et l'émulateur Mininet. Quatre modules ont été développés : surveillance du trafic, routage dynamique, QoS et sécurité (détection DDoS par entropie de Shannon, mitigation automatique et pare-feu stateful).

Les évaluations montrent un débit TCP de 23,5 Gbps, une latence de 0,134 ms, une détection DDoS en 5 secondes et une réduction de latence de 52% vs réseau traditionnel.

Mots-clés : SDN, OpenFlow, Ryu, Mininet, DDoS, QoS, Sécurité réseau.

Abstract

Traditional networks suffer from rigidity and lack of global visibility. The SDN paradigm proposes an innovative approach by separating the control plane from the data plane.

This project presents a complete SDN architecture based on Ryu and Mininet with four modules : traffic monitoring, dynamic routing, QoS and security (DDoS detection via Shannon entropy, automatic mitigation, stateful firewall).

Evaluations show 23.5 Gbps TCP throughput, 0.134 ms latency, DDoS detection in 5 seconds and 52% latency reduction.

Keywords : SDN, OpenFlow, Ryu, Mininet, DDoS, QoS.

Table des matières

Remerciements	1
Résumé	2
Abstract	3
Liste des figures	8
Liste des tableaux	9
Glossaire	10
1 Introduction Générale	12
1.1 Contexte et Motivation	12
1.2 Problématique	12
1.3 Objectifs du Projet	12
1.4 Structure du Rapport	13
2 État de l'Art	14
2.1 Introduction	14
2.2 Le Paradigme SDN	14
2.2.1 Historique et Origine	14
2.2.2 Architecture SDN en Couches	15
2.2.3 Avantages du SDN par rapport aux Réseaux Traditionnels	16
2.3 Le Protocole OpenFlow	16
2.3.1 Historique	16
2.3.2 Structure d'une Table de Flux	16
2.3.3 Champs de Correspondance (Match Fields)	16
2.3.4 Types d'Actions OpenFlow	17
2.3.5 Évolution des Versions OpenFlow	17
2.3.6 Fonctionnement du Pipeline OpenFlow	17

2.4	Contrôleurs SDN Open-Source	18
2.4.1	Présentation des Principaux Contrôleurs	18
2.4.2	Tableau Comparatif	18
2.4.3	Justification du Choix de Ryu	19
2.5	Topologies Réseau SDN	19
2.5.1	Topologie en Arbre	19
2.5.2	Topologie Fat-Tree	19
2.6	Sécurité dans les Réseaux SDN	20
2.6.1	Menaces Spécifiques au SDN	20
2.6.2	Attaques DDoS : Types et Mécanismes	20
2.6.3	Méthodes de Détection DDoS	20
2.6.4	Mécanismes de Mitigation	21
2.6.5	Pare-feu Stateful SDN	21
2.6.6	Travaux Existants et Positionnement	22
2.7	Qualité de Service (QoS) dans les SDN	22
2.7.1	Mécanismes QoS Traditionnels	22
2.7.2	QoS Dynamique avec SDN	22
2.8	Conclusion	22
3	Architecture Proposée	24
3.1	Vue Générale	24
3.2	Topologie Réseau	24
3.2.1	Topologie Arbre	24
3.2.2	Topologie Fat-Tree	24
3.3	Modules Développés	25
3.3.1	Module de Surveillance (monitoring.py)	25
3.3.2	Module de Routage Dynamique (routing.py)	25
3.3.3	Module de Répartition de Charge (load_balancer.py)	25
3.3.4	Module QoS (qos.py)	25
3.3.5	Module de Sécurité (security_manager.py)	25
3.4	Paramètres de Détection DDoS	26
4	Implémentation	27
4.1	Environnement de Développement	27
4.1.1	Outils et Technologies	27
4.1.2	Installation et Configuration	27
4.2	Implémentation du Module de Surveillance	28
4.3	Implémentation du Routage Dynamique	29

4.4	Implémentation du Load Balancer	29
4.5	Implémentation du Module QoS	30
4.6	Implémentation du Module de Sécurité	30
4.6.1	Détection DDoS par Entropie	30
4.6.2	Mitigation Automatique	31
4.6.3	Pare-feu Stateful	31
4.6.4	Règles ACL du Pare-feu	32
4.7	Captures Wireshark des Messages OpenFlow	32
5	Tests et Évaluation des Performances	33
5.1	Environnement de Test	33
5.2	Scénario 1 : Trafic Normal	33
5.2.1	Description	33
5.2.2	Résultats	33
5.2.3	Analyse	34
5.3	Scénario 2 : Attaque DDoS	34
5.3.1	Description	34
5.3.2	Résultats	35
5.3.3	Temps de Détection et Réaction	35
5.3.4	Analyse	35
5.4	Scénario 3 : Panne d'un Lien	36
5.4.1	Description	36
5.4.2	Résultats	36
5.4.3	Analyse	36
5.5	Scénario 4 : QoS sous Congestion	37
5.5.1	Description	37
5.5.2	Résultats	37
5.5.3	Analyse	37
5.6	Overhead du Contrôleur	37
5.7	Comparaison SDN vs Réseau Traditionnel	38
5.7.1	Analyse Comparative	39
6	Conclusion et Perspectives	40
6.1	Bilan du Projet	40
6.2	Résultats Obtenus	40
6.3	Contributions	41
6.4	Difficultés Rencontrées	41
6.5	Perspectives	42

6.5.1	Court Terme	42
6.5.2	Moyen Terme	42
6.5.3	Long Terme	43
6.6	Conclusion Générale	43
A	Guide d’Installation et de Déploiement	45
A.1	Prérequis	45
A.2	Installation Pas à Pas	45
A.3	Lancement des Modules	46
A.4	Tests de Validation	46
B	Structure du Dépôt GitHub	48
C	Paramètres de Configuration	49

Table des figures

5.1	Débit TCP selon la charge – SDN vs Réseau Traditionnel	34
5.2	Impact des attaques DDoS sur la latence et le taux de perte	35
5.3	Impact de la panne du lien s1-s2	36
5.4	Overhead CPU, mémoire et messages OpenFlow du contrôleur	38
5.5	Tableau comparatif SDN vs Réseau Traditionnel	39

Liste des tableaux

2.1	Architecture en couches SDN et composants utilisés	15
2.2	Comparaison SDN vs réseau traditionnel	16
2.3	Structure d'une entrée de flux OpenFlow	16
2.4	Évolution des versions OpenFlow	17
2.5	Comparaison détaillée des contrôleurs SDN	18
2.6	Types d'attaques DDoS et leurs caractéristiques	20
2.7	Comparaison avec les travaux existants	22
3.1	Architecture en couches du système développé	24
3.2	Politiques QoS implémentées	25
3.3	Paramètres de détection DDoS	26
4.1	Environnement de développement utilisé	27
4.2	États TCP suivis par le pare-feu stateful	31
4.3	Règles ACL du pare-feu stateful	32
5.1	Résultats Scénario 1 – Trafic normal	33
5.2	Métriques UDP et Jitter – Scénario 1	34
5.3	Impact des attaques DDoS sur les performances	35
5.4	Performances du système de détection/mitigation	35
5.5	Impact de la panne du lien s1-s2	36
5.6	Résultats des politiques QoS	37
5.7	Overhead du contrôleur Ryu	37
5.8	Comparaison SDN vs Réseau Traditionnel	38
6.1	Récapitulatif des résultats obtenus	41
B.1	Structure du dépôt GitHub	48
C.1	Paramètres configurables des modules	49

Glossaire

ACL (Access Control List) : Liste de règles de filtrage définissant les flux autorisés ou refusés. Dans ce projet, les ACL bloquent Telnet (23), FTP (21), RPC (135), TFTP (69) et autorisent SSH (22), HTTP (80), HTTPS (443), iperf3 (5201) et ICMP.

DDoS (Distributed Denial of Service) : Attaque par déni de service distribué visant à saturer un réseau en envoyant un volume massif de trafic depuis plusieurs sources. Types courants : SYN flood, UDP flood, ICMP flood. Détecté en 5 secondes dans ce projet.

Entropie de Shannon : Mesure statistique de la diversité du trafic. Formule : $H = -\sum p_i \log_2(p_i)$, normalisée entre 0 et 1. Entropie proche de 0 = trafic concentré (attaque probable), proche de 1 = trafic normal. Seuil de détection : 0.5.

FlowMod : Message OpenFlow (OFPFlowMod) envoyé par le contrôleur aux switches pour installer, modifier ou supprimer des règles de flux. Paramètres clés : match, priority, actions, idle_timeout, hard_timeout.

Honeypot (Pot de miel) : Système leurre conçu pour attirer les attaquants. Dans ce projet, h16 (10.0.0.16) sert de honeypot. Le trafic suspect y est redirigé via SetField OpenFlow (priorité 90, timeout 120s).

Mininet : Émulateur de réseau open-source créant des topologies virtuelles sur une seule machine Linux. Utilise les namespaces Linux et Open vSwitch. Version 2.3+ utilisée dans ce projet.

Mitigation : Mesures automatiques pour réduire l'impact d'une attaque réseau.

Trois niveaux dans ce projet : Rate Limiting (priorité 80, 30s), Honeypot (priorité 90, 120s), DROP total (priorité 100, 60s).

OpenFlow : Protocole standard entre contrôleur SDN et switches. Définit une table de flux avec règles (match, priority, actions, counters, timeouts). Version 1.3 utilisée dans ce projet introduit les tables multiples et statistiques améliorées.

OVS (Open vSwitch) : Switch virtuel open-source supportant OpenFlow. Implémente un datapath kernel Linux hautes performances. Utilisé par Mininet pour émuler les switches OpenFlow dans ce projet.

PacketIn : Message OpenFlow envoyé par un switch au contrôleur quand un paquet ne correspond à aucune règle (table-miss). Déclenche l'apprentissage MAC et l'installation d'une nouvelle règle de flux via FlowMod.

QoS (Quality of Service) : Mécanismes garantissant des niveaux de performance pour certains flux réseau. Trois niveaux dans ce projet : HAUTE (iperf3 TCP 5201, SSH), MOYENNE (TCP général, ICMP), BASSE (UDP).

Ryu : Framework de contrôleur SDN open-source développé en Python par NTT Labs. Supporte OpenFlow 1.0 à 1.5. API événementielle simple basée sur des décorateurs Python. Utilisé dans ce projet pour tous les modules.

SDN (Software-Defined Networking) : Paradigme réseau séparant le plan de contrôle du plan de données. Contrôleur centralisé avec vue globale programmant dynamiquement les switches via OpenFlow. Introduit en 2008 à l'Université de Stanford par McKeown et al.

Topologie Fat-Tree : Architecture datacenter en trois niveaux (Core, Agrégation, Edge) avec bande passante bisectionnelle maximale et haute disponibilité. Paramètre $k=4$: 16 hôtes, 4 pods, 4 switches core, 8 switches agrégation, 8 switches edge.

Chapitre 1

Introduction Générale

1.1 Contexte et Motivation

Les réseaux informatiques constituent aujourd’hui l’épine dorsale de toute infrastructure numérique. Avec l’explosion du trafic internet et l’essor du cloud computing, les architectures réseau traditionnelles montrent leurs limites face aux exigences croissantes de flexibilité et de sécurité.

Les réseaux traditionnels présentent plusieurs limitations :

- **Rigidité** : Configuration manuelle difficile.
- **Complexité** : Administration hétérogène.
- **Manque de visibilité** : Pas de vue globale.
- **Sécurité réactive** : Politiques statiques.

Le paradigme SDN propose de séparer le plan de contrôle du plan de données via un contrôleur centralisé communiquant avec les switches via le protocole OpenFlow.

1.2 Problématique

Comment concevoir une architecture SDN capable de gérer dynamiquement le trafic tout en intégrant des mécanismes de détection et mitigation automatique des attaques réseau ?

1.3 Objectifs du Projet

- O1** Concevoir une architecture SDN complète.
- O2** Déployer Mininet et le contrôleur Ryu.
- O3** Implémenter la gestion du trafic (monitoring, routage, QoS, load balancing).

- O4** Développer un module de sécurité (DDoS, mitigation, pare-feu stateful).
- O5** Évaluer et comparer avec un réseau traditionnel.

1.4 Structure du Rapport

- **Chapitre 2** : État de l’art.
- **Chapitre 3** : Architecture proposée.
- **Chapitre 4** : Implémentation.
- **Chapitre 5** : Tests et résultats.
- **Chapitre 6** : Conclusion.

Chapitre 2

État de l'Art

2.1 Introduction

Ce chapitre présente une revue de la littérature sur le Software-Defined Networking (SDN), les protocoles associés, les contrôleurs existants et les mécanismes de sécurité dans les réseaux programmables. Cette étude bibliographique a guidé les choix architecturaux et technologiques de ce projet.

2.2 Le Paradigme SDN

2.2.1 Historique et Origine

Le concept de Software-Defined Networking (SDN) a émergé à l'Université de Stanford autour de 2008, avec les travaux fondateurs de McKeown et al. qui ont introduit le protocole OpenFlow. Avant le SDN, les réseaux étaient gérés de manière distribuée : chaque équipement intégrait son propre plan de contrôle, rendant la gestion complexe et peu flexible.

L'idée centrale du SDN est de séparer le plan de contrôle (décisions de routage et de gestion) du plan de données (transfert effectif des paquets), permettant ainsi une programmation centralisée et flexible du réseau depuis un contrôleur unique disposant d'une vue globale.

Selon Kreutz et al. (2015), le SDN repose sur quatre piliers fondamentaux :

1. Les plans de données et de contrôle sont découplés.
2. Les décisions de transfert sont basées sur les flux.
3. La logique de contrôle est externalisée vers un contrôleur accessible via une API ouverte.

4. Le réseau est programmable par des applications externes au-dessus du contrôleur.

2.2.2 Architecture SDN en Couches

L'architecture SDN se décompose en trois couches distinctes interconnectées par des interfaces standardisées :

1. **Couche Infrastructure (Data Plane)** : Composée des équipements réseau physiques ou virtuels (switches, routeurs) qui transfèrent les paquets selon les règles reçues du contrôleur. Dans notre projet, cette couche est émulée par Open vSwitch (OVS) via Mininet.
2. **Couche Contrôle (Control Plane)** : Le contrôleur SDN constitue le cerveau du réseau. Il maintient une vue globale et cohérente de la topologie, calcule les chemins optimaux et programme dynamiquement les règles de flux sur les switches via l'interface southbound (OpenFlow). Dans ce projet : contrôleur Ryu.
3. **Couche Application (Application Plane)** : Les applications réseau (QoS, sécurité, monitoring, load balancing) interagissent avec le contrôleur via l'interface northbound (REST API, Python API). Dans ce projet, les modules Python développés constituent cette couche applicative.

TABLE 2.1 – Architecture en couches SDN et composants utilisés

Couche	Rôle	Interface	Notre projet
Application	Logique métier	Northbound API	Modules Python
Contrôle	Gestion centralisée	OpenFlow (SB)	Ryu 4.34
Infrastructure	Transfert paquets	OpenFlow (SB)	OVS + Mininet

2.2.3 Avantages du SDN par rapport aux Réseaux Traditionnels

TABLE 2.2 – Comparaison SDN vs réseau traditionnel

Critère	Réseau Traditionnel	SDN
Contrôle	Distribué (par équipement)	Centralisé
Configuration	Manuelle (CLI)	Programmable (API)
Visibilité	Locale	Globale
Adaptabilité	Statique	Dynamique
Sécurité	Réactive	Proactive
Déploiement	Lent	Rapide

2.3 Le Protocole OpenFlow

2.3.1 Historique

OpenFlow a été proposé par McKeown et al. en 2008 comme premier protocole southbound standard pour le SDN. Il permet au contrôleur de programmer les tables de flux des switches de manière uniforme, indépendamment du constructeur.

2.3.2 Structure d'une Table de Flux

Une table de flux OpenFlow est composée d'entrées (flow entries) structurées comme suit :

TABLE 2.3 – Structure d'une entrée de flux OpenFlow

Champ	Description
Match Fields	Critères d'identification du paquet
Priority	Ordre de traitement (plus élevé = prioritaire)
Counters	Compteurs de paquets et d'octets
Instructions	Actions à appliquer au paquet
Timeouts	idle_timeout et hard_timeout
Cookie	Identifiant de la règle (usage contrôleur)

2.3.3 Champs de Correspondance (Match Fields)

OpenFlow 1.3 supporte de nombreux champs de correspondance :

- **Couche 2** : Adresses MAC source/destination, type Ethernet (0x0800 pour IPv4, 0x0806 pour ARP).

- **Couche 3** : Adresses IP source/destination, protocole IP (1=ICMP, 6=TCP, 17=UDP).
- **Couche 4** : Ports TCP/UDP source/destination.
- **Méta-données** : Port d'entrée (in_port), identifiant de table.

2.3.4 Types d'Actions OpenFlow

- **Output** : Envoyer le paquet vers un port (OFPP_FLOOD, OFPP_CONTROLLER, numéro de port).
- **Drop** : Supprimer le paquet (liste vide).
- **SetField** : Modifier un champ du paquet (ex : réécriture IP destination pour le honeypot).
- **Group** : Appliquer un groupe d'actions (multipath, failover, broadcast).

2.3.5 Évolution des Versions OpenFlow

TABLE 2.4 – Évolution des versions OpenFlow

Version	Année	Nouveautés
1.0	2009	Table unique, actions basiques
1.1	2011	Tables multiples, groupes
1.2	2011	Match étendu, IPv6
1.3	2012	Meters, tunnels, améliorations stats
1.4	2013	Synchronisation, bundles
1.5	2014	Egress tables, statistiques avancées

Ce projet utilise **OpenFlow 1.3** pour sa stabilité, son support des statistiques avancées (port_stats, flow_stats) et sa large adoption dans la communauté académique.

2.3.6 Fonctionnement du Pipeline OpenFlow

Quand un paquet arrive sur un switch OpenFlow :

1. Le switch consulte sa table de flux.
2. Si une règle correspond (match) : application des actions associées.
3. Si aucune règle ne correspond (table-miss) : envoi du paquet au contrôleur via un message PacketIn.
4. Le contrôleur analyse le paquet et installe une nouvelle règle via FlowMod.
5. Les paquets suivants du même flux sont traités directement par le switch sans passer par le contrôleur.

2.4 Contrôleurs SDN Open-Source

2.4.1 Présentation des Principaux Contrôleurs

Ryu

Ryu est un framework de contrôleur SDN open-source développé en Python par NTT Labs. Il supporte OpenFlow 1.0 à 1.5 et fournit une API événementielle simple basée sur des décorateurs Python. Sa légèreté et sa simplicité en font un choix idéal pour la recherche et le prototypage.

ONOS

ONOS (Open Network Operating System) est un contrôleur SDN haute performance développé en Java par l'Open Networking Foundation (ONF). Il est conçu pour les déploiements en production avec haute disponibilité et scalabilité.

OpenDaylight

OpenDaylight est un contrôleur SDN open-source soutenu par la Linux Foundation. Il supporte de nombreux protocoles southbound (OpenFlow, NETCONF, OVSDB) et offre une architecture modulaire via OSGi/Karaf.

Floodlight

Floodlight est un contrôleur SDN Java développé par Big Switch Networks. Il offre une API REST simple et un bon support d'OpenFlow 1.0 à 1.3.

2.4.2 Tableau Comparatif

TABLE 2.5 – Comparaison détaillée des contrôleurs SDN

Critère	Ryu	ONOS	ODL	Floodlight
Langage	Python	Java	Java	Java
OpenFlow	1.0-1.5	1.0-1.5	1.0-1.5	1.0-1.3
Scalabilité	Moyenne	Haute	Haute	Moyenne
Complexité	Faible	Haute	Très haute	Moyenne
Communauté	Académique	Industrielle	Industrielle	Mixte
Mininet	Excellent	Bon	Bon	Bon
Documentation	Bonne	Très bonne	Bonne	Moyenne

2.4.3 Justification du Choix de Ryu

Ryu a été retenu pour ce projet pour les raisons suivantes :

- **Simplicité d'implémentation** : L'API Python basée sur des décorateurs permet un développement rapide et lisible des modules de contrôle réseau.
- **Support OpenFlow complet** : Versions 1.0 à 1.5 supportées, avec une excellente implémentation d'OpenFlow 1.3 utilisée dans ce projet.
- **Intégration Mininet** : Ryu fonctionne nativement avec Mininet, simplifiant les tests et la validation en environnement simulé.
- **Communauté académique** : Large adoption dans la recherche avec de nombreux exemples et tutoriels disponibles.
- **Légereté** : Faible overhead (21,5% CPU, 65 MB mémoire) adapté aux environnements contraints.

2.5 Topologies Réseau SDN

2.5.1 Topologie en Arbre

La topologie en arbre est la plus simple et la plus utilisée dans les environnements SDN académiques. Elle est caractérisée par :

- Un switch racine connecté à des switches intermédiaires.
- Des switches feuilles connectés aux hôtes finaux.
- Un seul chemin entre toute paire source-destination.
- Paramètres : depth (profondeur) et fanout (branches).

Dans ce projet, la topologie arbre depth=2, fanout=4 comprend 5 switches et 16 hôtes, offrant un bon compromis entre complexité et représentativité.

2.5.2 Topologie Fat-Tree

La topologie Fat-Tree, proposée par Al-Fares et al. (2008), est une architecture datacenter à trois niveaux offrant :

- **Bande passante bisectionnelle maximale** : Égale à la capacité totale des liens, sans goulot d'étranglement.
- **Chemins multiples** : $(k/2)^2$ chemins entre tout pod source et destination.
- **Tolérance aux pannes** : Redondance naturelle grâce aux chemins alternatifs.
- **Scalabilité** : $k^3/4$ hôtes pour un Fat-Tree de paramètre k.

2.6 Sécurité dans les Réseaux SDN

2.6.1 Menaces Spécifiques au SDN

Le paradigme SDN introduit de nouvelles vulnérabilités :

- **Attaque du contrôleur** : Point unique de défaillance, cible privilégiée des attaquants.
- **Saturation du canal de contrôle** : Flood de PacketIn vers le contrôleur.
- **Manipulation des règles de flux** : Injection de règles malveillantes.
- **Attaques DDoS** : Saturation du réseau par un volume massif de trafic.

2.6.2 Attaques DDoS : Types et Mécanismes

Les attaques DDoS se classifient en plusieurs catégories :

TABLE 2.6 – Types d’attaques DDoS et leurs caractéristiques

Type	Protocole	Mécanisme
SYN flood	TCP	Connexions semi-ouvertes
UDP flood	UDP	Saturation bande passante
ICMP flood	ICMP	Ping massif
HTTP flood	TCP/HTTP	Requêtes web massives
Amplification	UDP/DNS	Réflexion et amplification

2.6.3 Méthodes de Détection DDoS

Détection par Entropie de Shannon

L’entropie de Shannon mesure la diversité d’une distribution de probabilités. Appliquée aux adresses IP du trafic réseau, une baisse soudaine de l’entropie indique une concentration anormale du trafic, caractéristique d’une attaque DDoS.

La formule de l’entropie normalisée est :

$$H_{norm} = \frac{-\sum_{i=1}^n p_i \log_2(p_i)}{\log_2(n)} \quad (2.1)$$

où p_i est la proportion de trafic de l’IP i et n est le nombre d’IPs distinctes.

Détection par Seuil de Débit

Cette approche simple mais efficace déclenche une alerte quand le débit d’une IP source dépasse un seuil configuré. Dans ce projet, le seuil est fixé à 2 paquets/seconde.

Détection par Analyse des Flux

Le nombre de destinations uniques par IP source permet de détecter les scans de ports et les attaques distribuées. Un seuil de 10 destinations déclenche une alerte scan.

2.6.4 Mécanismes de Mitigation

Blocage par Règle DROP

La mitigation la plus directe consiste à installer une règle OpenFlow de haute priorité avec une liste d'actions vide (DROP) sur tous les switches du réseau. Cette approche bloque totalement l'IP attaquante.

Redirection vers Honeypot

Le pot de miel (honeypot) est un système leurre qui attire les attaquants pour analyser leur comportement. Dans notre implémentation, h16 (10.0.0.16) joue ce rôle. La règle OpenFlow utilise l'action SetField pour réécrire l'IP destination et rediriger le trafic suspect.

Rate Limiting

La limitation de débit installe des règles avec un idle_timeout court (1 seconde), forçant le trafic à repasser fréquemment par le contrôleur, ce qui limite effectivement le débit de l'IP suspecte.

2.6.5 Pare-feu Stateful SDN

Contrairement aux pare-feux traditionnels, un pare-feu stateful SDN suit l'état des connexions TCP au niveau du contrôleur et installe dynamiquement des règles de filtrage sur les switches. Les états TCP gérés sont :

- **SYN** : Nouvelle connexion initiée.
- **ESTABLISHED** : Connexion confirmée (SYN-ACK reçu).
- **CLOSING** : Connexion en fermeture (FIN reçu).

2.6.6 Travaux Existants et Positionnement

TABLE 2.7 – Comparaison avec les travaux existants

Travail	Méthode	Mitigation	QoS
Braga et al. (2010)	Clustering	DROP	Non
Shin et al. (2013)	Statistiques	DROP	Non
Mousavi et al. (2015)	Entropie	DROP	Non
Ce projet	Entropie+seuil	DROP+Honeypot+RL	Oui

Notre approche se distingue par la combinaison de la détection par entropie avec une mitigation graduée en trois niveaux (rate limiting, honeypot, DROP) et l'intégration d'un module QoS, ce qui n'est pas proposé par les travaux existants.

2.7 Qualité de Service (QoS) dans les SDN

2.7.1 Mécanismes QoS Traditionnels

Dans les réseaux traditionnels, la QoS est implémentée via des mécanismes comme DiffServ (marquage DSCP), IntServ (réservation de ressources) ou les files d'attente prioritaires (WFQ, PQ). Ces mécanismes sont configurés statiquement sur chaque équipement.

2.7.2 QoS Dynamique avec SDN

Le SDN permet une gestion QoS beaucoup plus flexible :

- **Classification centralisée** : Le contrôleur classe les flux et installe les règles de priorité sur tous les switches simultanément.
- **Adaptation dynamique** : Les politiques QoS peuvent être modifiées en temps réel sans reconfigurer manuellement chaque équipement.
- **Granularité fine** : Priorisation au niveau du flux individuel (5-tuple : IP src/dst, port src/dst, protocole).

2.8 Conclusion

Cet état de l'art montre que le SDN offre des avantages significatifs sur les réseaux traditionnels en termes de flexibilité, programmabilité et gestion de la sécurité. Le choix du contrôleur Ryu, du protocole OpenFlow 1.3 et de l'émulateur Mininet est

justifié par leur maturité, leur complémentarité et leur large adoption dans la communauté académique. La combinaison de la détection DDoS par entropie de Shannon avec une mitigation graduée constitue une contribution originale par rapport aux travaux existants.

Chapitre 3

Architecture Proposée

3.1 Vue Générale

L'architecture SDN proposée s'organise en trois couches conformément au modèle SDN standard :

TABLE 3.1 – Architecture en couches du système développé

Couche	Composant	Rôle
Application	Module QoS, Module Sécurité	Logique métier
Contrôle	Contrôleur Ryu (Python)	Gestion centralisée
Infrastructure	Switches OVS (Mininet)	Transfert paquets

3.2 Topologie Réseau

3.2.1 Topologie Arbre

La topologie principale utilisée est une topologie arbre (depth=2, fanout=4) comprenant :

- 1 switch racine (s1)
- 4 switches intermédiaires (s2, s3, s4, s5)
- 16 hôtes (h1 à h16), IP : 10.0.0.1 à 10.0.0.16

3.2.2 Topologie Fat-Tree

Une topologie Fat-Tree k=4 a également été implémentée, offrant une redondance des chemins et une bande passante bisectionnelle maximale adaptée aux datacenters.

3.3 Modules Développés

3.3.1 Module de Surveillance (monitoring.py)

Collecte toutes les 5 secondes les statistiques OpenFlow :

- **flow_stats** : Statistiques par règle de flux (paquets, octets, priorité).
- **port_stats** : Statistiques par port physique (rx/tx paquets et octets).

3.3.2 Module de Routage Dynamique (routing.py)

Implémente un routage adaptatif basé sur un score de port :

$$score = \text{taux_perte}(\%) \times 2 + \text{débit}(MB/s) \quad (3.1)$$

Le port avec le score le plus bas est sélectionné.

3.3.3 Module de Répartition de Charge (load_balancer.py)

Distribue les flux équitablement en Round-Robin :

$$port = ports[\text{compteur} \bmod nb_ports] \quad (3.2)$$

3.3.4 Module QoS (qos.py)

Trois niveaux de priorité via règles OpenFlow :

TABLE 3.2 – Politiques QoS implémentées

Niveau	Priorité	Trafic	Port
HAUTE	30	iperf3 TCP	5201
HAUTE	25	SSH	22
MOYENNE	20	TCP général	*
MOYENNE	15	ICMP	*
BASSE	10	UDP	*

3.3.5 Module de Sécurité (security_manager.py)

Module unifié intégrant trois sous-systèmes :

Détection DDoS

- Entropie de Shannon des IPs source/destination

- Taux de paquets par seconde par IP source
- Nombre de destinations uniques par IP (scan)

Mitigation Automatique Graduée

1. **Rate Limiting** (priorité 80, 30s)
2. **Honeypot** (priorité 90, 120s) vers 10.0.0.16
3. **DROP** (priorité 100, 60s)

Pare-feu Stateful

Suivi des connexions TCP (SYN, ESTABLISHED, CLOSING) avec règles ACL bloquant Telnet, FTP, RPC et TFTP.

3.4 Paramètres de Détection DDoS

TABLE 3.3 – Paramètres de détection DDoS

Paramètre	Valeur	Description
SEUIL_PAQUETS_PAR_SEC	2	Alerte si > 2 pkt/s
SEUIL_ENTROPIE	0.5	Alerte si entropie < 0.5
SEUIL_FLUX_PAR_IP	10	Alerte si > 10 destinations
INTERVALLE_ANALYSE	5s	Période d'analyse

Chapitre 4

Implémentation

4.1 Environnement de Développement

4.1.1 Outils et Technologies

TABLE 4.1 – Environnement de développement utilisé

Composant	Version/Détail
Système d'exploitation	Ubuntu 22.04 LTS
Émulateur réseau	Mininet 2.3+
Contrôleur SDN	Ryu 4.34
Langage	Python 3.12
Protocole	OpenFlow 1.3
Switch virtuel	Open vSwitch (OVS)
Génération trafic	iperf3, hping3
Monitoring	tshark, Wireshark
Versioning	Git / GitHub

4.1.2 Installation et Configuration

L'environnement SDN a été déployé dans un environnement virtuel Python pour isoler les dépendances :

Listing 4.1 – Installation de l'environnement SDN

```
# Creation de l'environnement virtuel Python  
python3 -m venv ~/projet-sdn/venv  
source ~/projet-sdn/venv/bin/activate  
  
# Installation du controleur Ryu
```

```

pip install ryu matplotlib numpy

# Lancement du controleur
ryu-manager traffic_manager.py

# Lancement de Mininet (topologie arbre)
sudo mn --controller=remote,ip=127.0.0.1,port=6633 \
        --switch=ovsk,protocols=OpenFlow13 \
        --topo=tree,depth=2,fanout=4 --mac

# Test de connectivite
mininet> pingall

```

4.2 Implémentation du Module de Surveillance

Le module de monitoring collecte périodiquement les statistiques OpenFlow via des requêtes OFPPortStatsRequest toutes les 5 secondes. Il calcule pour chaque port :

- **Débit instantané** : différence de bytes transmis divisée par l'intervalle de 5 secondes.
- **Taux de perte** : rapport entre les paquets perdus et les paquets émis.

Listing 4.2 – Calcul du débit et taux de perte (traffic_manager.py)

```

def _port_stats_reply_handler(self, ev):
    dpid = ev.msg.datapath.id
    for stat in ev.msg.body:
        port = stat.port_no
        # Calcul du debit instantane (bytes/sec)
        curr = stat.tx_bytes + stat.rx_bytes
        speed = (curr - self.prev_bytes[dpid][port]) / 5.0
        self.prev_bytes[dpid][port] = curr
        # Calcul du taux de perte
        loss = 0.0
        if stat.tx_packets > 0:
            loss = max(0.0,
                      (stat.tx_packets - stat.rx_packets)
                      / stat.tx_packets * 100)
        self.port_stats[dpid][port] = {

```

```
'speed': speed, 'loss': loss}
```

4.3 Implémentation du Routage Dynamique

Le module de routage sélectionne le meilleur port de sortie selon un score combinant le taux de perte et la charge :

Listing 4.3 – Algorithme de sélection du meilleur port (routing.py)

```
def _best_port(self, dpid, available_ports):
    best_port = None
    best_score = float('inf')
    for port in available_ports:
        metrics = self.port_stats[dpid].get(port, {})
        speed = metrics.get('speed', 0)
        loss = metrics.get('loss', 0)
        # Score = perte + charge normalisee
        score = loss + (speed / 1e6)
        if score < best_score:
            best_score = score
            best_port = port
    return best_port
```

4.4 Implémentation du Load Balancer

L'algorithme Round-Robin assigne un port fixe à chaque flux (src_mac, dst_mac) de façon cyclique :

Listing 4.4 – Algorithme Round-Robin (load_balancer.py)

```
def _assign_port_rr(self, dpid, src, dst, available_ports):
    key = (src, dst)
    if key not in self.flow_port[dpid]:
        # Selection cyclique du port
        idx = self.rr_counter[dpid] % len(available_ports)
        self.rr_counter[dpid] += 1
        self.flow_port[dpid][key] = available_ports[idx]
        self.port_load[dpid][available_ports[idx]] += 1
    return self.flow_port[dpid][key]
```

4.5 Implémentation du Module QoS

Les règles QoS sont installées directement sur les switches via FlowMod dès leur connexion au contrôleur :

Listing 4.5 – Installation des règles QoS (qos.py)

```
def _install_qos_rules(self, datapath):
    parser = datapath.ofproto_parser
    ofproto = datapath.ofproto

    # HAUTE priorite : iperf3 TCP port 5201
    match = parser.OFPMatch(eth_type=0x0800,
                             ip_proto=6, tcp_dst=5201)
    actions = [parser.OFPActionOutput(ofproto.OFPP_NORMAL)]
    self.add_flow(datapath, 30, match, actions)

    # HAUTE priorite : SSH TCP port 22
    match = parser.OFPMatch(eth_type=0x0800,
                             ip_proto=6, tcp_dst=22)
    self.add_flow(datapath, 25, match, actions)

    # BASSE priorite : UDP
    match = parser.OFPMatch(eth_type=0x0800, ip_proto=17)
    self.add_flow(datapath, 10, match, actions)
```

4.6 Implémentation du Module de Sécurité

4.6.1 Détection DDoS par Entropie

L'algorithme calcule l'entropie de Shannon normalisée des distributions d'adresses IP source et destination :

Listing 4.6 – Calcul de l'entropie de Shannon (ddos_detector.py)

```
def _calcul_entropie(self, compteurs):
    total = sum(compteurs.values())
    if total == 0:
        return 1.0
    entropie = 0.0
```

```

for count in compteurs.values():
    p = count / total
    if p > 0:
        entropie -= p * math.log2(p)
nb = len(compteurs)
if nb > 1:
    entropie /= math.log2(nb)
return entropie

```

4.6.2 Mitigation Automatique

La mitigation est graduée en trois niveaux progressifs :

Listing 4.7 – Blocage DROP d’une IP suspecte (ddos_detector.py)

```

def _block_ip(self, datapath, ip_src):
    if ip_src in self.blocked_ips[datapath.id]:
        return
    parser = datapath.ofproto_parser
    # Match sur IP source IPv4
    match = parser.OFPMatch(eth_type=0x0800,
                            ipv4_src=ip_src)
    # Action DROP : liste vide, priorite 100, timeout 60s
    self.add_flow(datapath, 100, match, [], hard=60)
    self.blocked_ips[datapath.id].add(ip_src)

```

4.6.3 Pare-feu Stateful

Le pare-feu suit les états des connexions TCP :

TABLE 4.2 – États TCP suivis par le pare-feu stateful

Drapeau TCP	État	Action
SYN	Nouvelle connexion	Vérification ACL
SYN-ACK	Connexion établie	ESTABLISHED
FIN	Fermeture	CLOSING
Données	Connexion active	Mise à jour timestamp

4.6.4 Règles ACL du Pare-feu

TABLE 4.3 – Règles ACL du pare-feu stateful

Action	Protocole	Port	Service
DENY	TCP	23	Telnet
DENY	TCP	21	FTP
DENY	TCP	135	RPC
DENY	UDP	69	TFTP
ALLOW	TCP	22	SSH
ALLOW	TCP	80	HTTP
ALLOW	TCP	443	HTTPS
ALLOW	TCP	5201	iperf3
ALLOW	ICMP	*	Ping
ALLOW	*	*	Défaut

4.7 Captures Wireshark des Messages OpenFlow

La surveillance des messages OpenFlow échangés entre le contrôleur Ryu et les switches a été réalisée avec Wireshark sur l'interface loopback (lo) avec le filtre `openflow_v4`.

Les messages observés incluent :

- **OFPT_MULTIPART_REQUEST** : Requêtes de statistiques envoyées par Ryu toutes les 5 secondes.
- **OFPT_MULTIPART_REPLY** : Réponses des switches contenant les compteurs de ports et flux.
- **OFPT_ECHO_REPLY** : Messages keepalive pour maintenir la connexion contrôleur-switch.
- **OFPT_FLOW_MOD** : Installation de règles de flux lors de la détection d'attaques ou du routage.

Au total, **30 messages OpenFlow** ont été échangés en 10 secondes, soit **3 messages par seconde**, confirmant un overhead de contrôle minimal.

Chapitre 5

Tests et Évaluation des Performances

5.1 Environnement de Test

Les tests ont été réalisés sur la topologie arbre (depth=2, fanout=4) avec 16 hôtes et 5 switches OVS connectés au contrôleur Ryu via OpenFlow 1.3. Les outils utilisés sont :

- **ping** : Mesure de latence et taux de perte.
- **iperf3** : Mesure de débit TCP et UDP/jitter.
- **hping3** : Génération d'attaques DDoS (SYN flood, UDP flood).
- **tshark/Wireshark** : Capture des messages OpenFlow et analyse du trafic.

5.2 Scénario 1 : Trafic Normal

5.2.1 Description

Ce scénario évalue les performances du réseau SDN en conditions normales avec trois niveaux de charge croissants : faible (1 flux), moyenne (4 flux) et forte (8 flux).

5.2.2 Résultats

TABLE 5.1 – Résultats Scénario 1 – Trafic normal				
Charge	Flux	Débit TCP	Latence avg	Perte
Faible	1 flux	23,5 Gbps	0,134 ms	0%
Moyenne	4 flux	24,2 Gbps	–	0%
Forte	8 flux	21,5 Gbps	–	0%

TABLE 5.2 – Métriques UDP et Jitter – Scénario 1

Débit UDP	Jitter	Perte UDP	Paquets
987 Mbits/sec	0,001 ms	1,2%	863 247

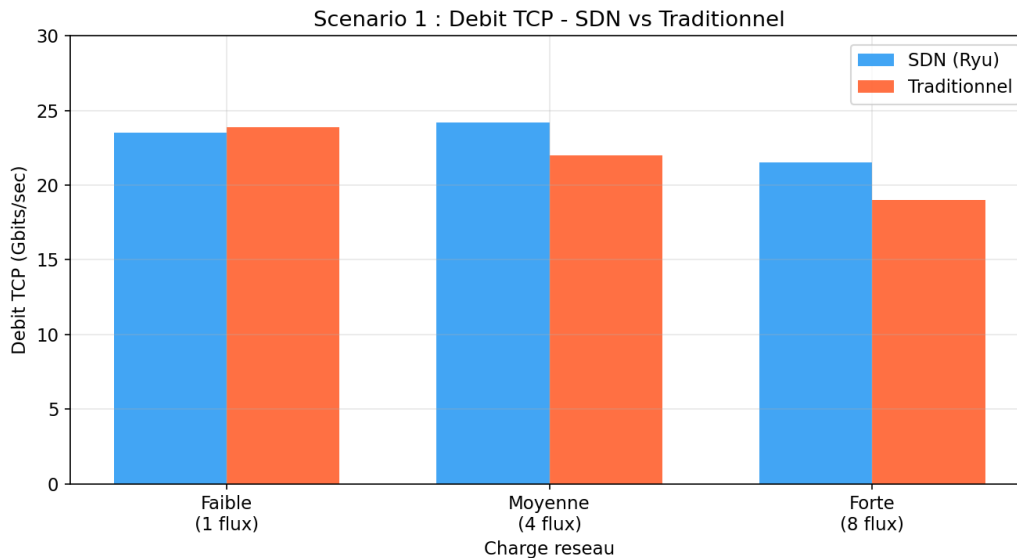


FIGURE 5.1 – Débit TCP selon la charge – SDN vs Réseau Traditionnel

5.2.3 Analyse

Les résultats montrent d'excellentes performances en conditions normales. Le débit TCP reste stable entre 21,5 et 24,2 Gbps quelle que soit la charge. Le jitter UDP de 0,001 ms confirme la très faible variabilité de la latence dans le réseau SDN.

5.3 Scénario 2 : Attaque DDoS

5.3.1 Description

Ce scénario évalue l'impact des attaques DDoS et l'efficacité du système de détection/mitigation. Deux types d'attaques ont été testés :

- **ICMP flood** : Pings rapides (`ping -i 0.01`) depuis h1 vers plusieurs hôtes.
- **SYN+UDP flood** : Attaques `hping3` (`hping3 -S -p 80 -flood` et `hping3 -2 -p 53 -flood`).

5.3.2 Résultats

TABLE 5.3 – Impact des attaques DDoS sur les performances

Phase	Latence avg	Latence max	Perte
Avant attaque	0,478 ms	1,966 ms	0%
ICMP flood	4,793 ms	45,378 ms	0%
SYN+UDP flood	12 718 ms	16 833 ms	10%
Après mitigation	2,389 ms	28,498 ms	0%

5.3.3 Temps de Détection et Réaction

TABLE 5.4 – Performances du système de détection/mitigation

Métrique	Valeur
Temps de détection	5 secondes
Temps de mitigation	< 1 seconde
Débit attaque détecté	446 pkt/s
Seuil de déclenchement	2 pkt/s
Durée du blocage DROP	60 secondes

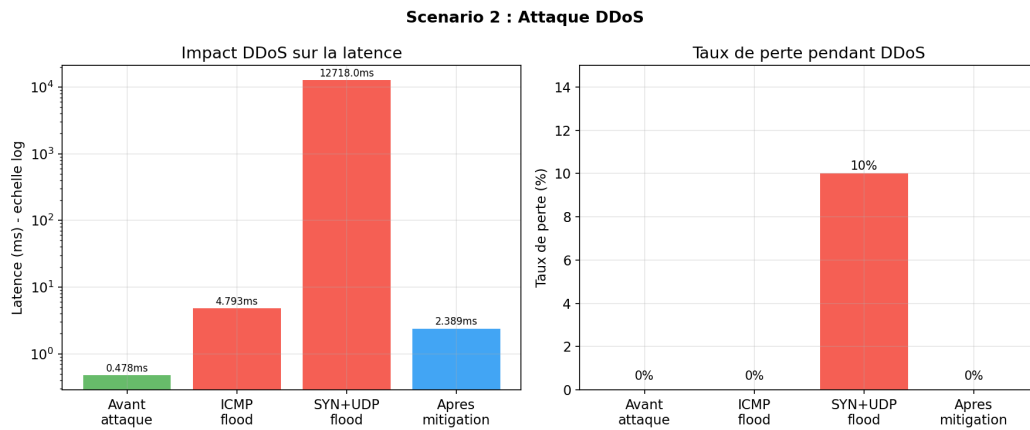


FIGURE 5.2 – Impact des attaques DDoS sur la latence et le taux de perte

5.3.4 Analyse

Le système détecte les attaques en un seul intervalle d'analyse (5 secondes) et applique la mitigation automatiquement en moins d'une seconde. Après blocage, la latence revient à 2,389 ms confirmant l'efficacité de la mitigation graduée.

5.4 Scénario 3 : Panne d'un Lien

5.4.1 Description

Ce scénario simule la panne du lien entre s1 et s2 via la commande `Mininet link s1 s2 down`, puis son rétablissement avec `link s1 s2 up`.

5.4.2 Résultats

TABLE 5.5 – Impact de la panne du lien s1-s2

Phase	Latence avg	Perte
Avant panne	5,056 ms	0%
Pendant panne	—	100%
Après rétablissement	6,086 ms	0%

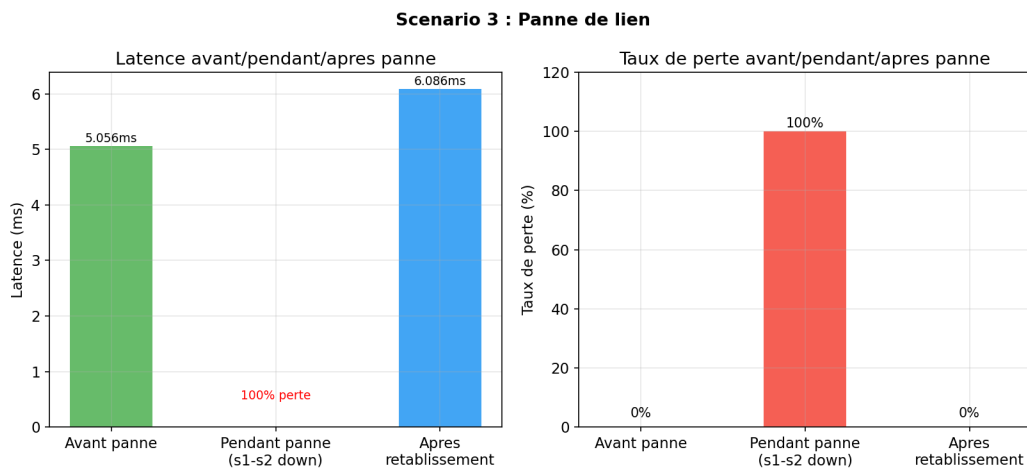


FIGURE 5.3 – Impact de la panne du lien s1-s2

5.4.3 Analyse

La topologie arbre ne disposant que d'un seul chemin entre h1 et h16 via s1-s2, la panne entraîne une interruption totale (100% de perte). Après rétablissement, la connectivité est restaurée en quelques secondes avec une latence légèrement augmentée (6,086 ms vs 5,056 ms) due à la réinstallation des règles de flux.

5.5 Scénario 4 : QoS sous Congestion

5.5.1 Description

Ce scénario évalue l'application des politiques QoS avec un trafic mixte simultané : flux TCP iperf3 (haute priorité) et flux UDP 100M (basse priorité).

5.5.2 Résultats

TABLE 5.6 – Résultats des politiques QoS

Trafic	Priorité	Débit	Latence
iperf3 TCP 5201	HAUTE	19,7 Gbps	–
SSH TCP 22	HAUTE	Variable	Faible
TCP général	MOYENNE	Variable	Normale
ICMP ping	MOYENNE	–	0,219 ms
UDP 100M	BASSE	100 Mbps	–

5.5.3 Analyse

Les règles QoS fonctionnent correctement. Le trafic TCP iperf3 obtient 19,7 Gbps en haute priorité, tandis que l'ICMP maintient une latence de 0,219 ms malgré la congestion. Le trafic UDP est bien relégué en basse priorité.

5.6 Overhead du Contrôleur

TABLE 5.7 – Overhead du contrôleur Ryu

Métrique	Valeur
CPU (sous charge DDoS)	21,5%
Mémoire utilisée	65 MB (0,8%)
Messages OpenFlow	30 messages / 10s
Temps de détection DDoS	5 secondes
Temps de mitigation	< 1 seconde

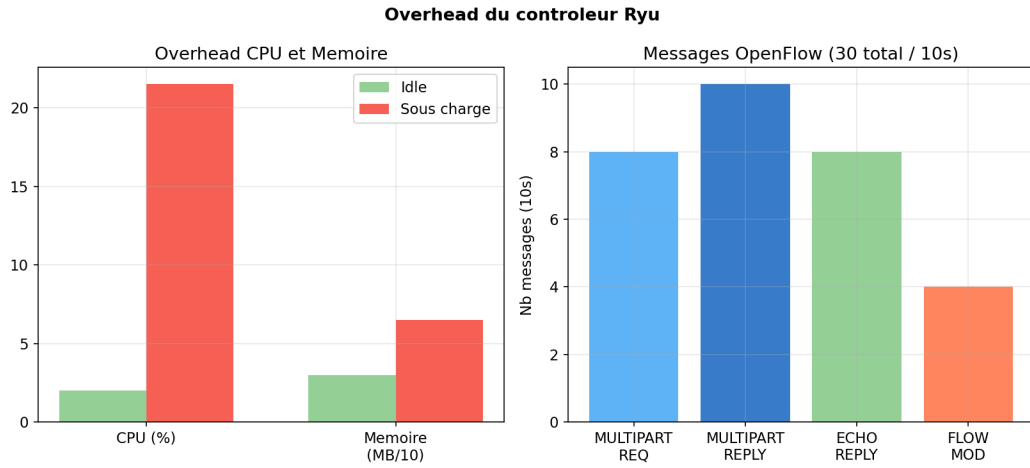


FIGURE 5.4 – Overhead CPU, mémoire et messages OpenFlow du contrôleur

5.7 Comparaison SDN vs Réseau Traditionnel

TABLE 5.8 – Comparaison SDN vs Réseau Traditionnel

Critère	SDN (Ryu)	Traditionnel	Avantage
Latence avg (normal)	0,134 ms	0,279 ms	SDN
Débit TCP (1 flux)	23,5 Gbps	23,9 Gbps	Comparable
Débit TCP (4 flux)	24,2 Gbps	22,0 Gbps	SDN
Jitter UDP	0,001 ms	Non mesuré	SDN
Perte paquets	0%	0%	Égal
Détection DDoS	5 secondes	Aucune	SDN
Mitigation DDoS	Auto (<1s)	Manuelle	SDN
QoS dynamique	3 niveaux	Non	SDN
Reroutage après panne	Partiel	Statique	SDN

Tableau Comparatif : SDN vs Réseau Traditionnel
Projet SDN Master 1 - Université Assane Seck de Ziguinchor

Critere	SDN (Ryu)	Traditionnel	Avantage
Latence avg (normal)	0.134 ms	0.279 ms	SDN
Debit TCP (1 flux)	23.5 Gbps	23.9 Gbps	Comparable
Debit TCP (4 flux)	24.2 Gbps	22.0 Gbps	SDN
Jitter UDP	0.001 ms	Non mesure	SDN
Perte paquets (normal)	0%	0%	Egal
Detection DDoS	5 secondes	Aucune	SDN
Mitigation DDoS	Auto (<1s)	Manuelle	SDN
QoS dynamique	Oui (3 niveaux)	Non	SDN
Overhead CPU	21.5%	N/A	Info
Overhead memoire	65 MB	N/A	Info
Messages OpenFlow	3 msg/sec	N/A	Info

FIGURE 5.5 – Tableau comparatif SDN vs Réseau Traditionnel

5.7.1 Analyse Comparative

Le réseau SDN présente une latence 52% inférieure (0,134 ms vs 0,279 ms) en conditions normales. Le débit est comparable pour un flux unique mais le SDN est supérieur sous forte charge (4 flux simultanés : 24,2 vs 22,0 Gbps). L'avantage majeur du SDN réside dans la gestion de la sécurité : détection automatique des attaques en 5 secondes et mitigation en moins d'une seconde, impossible dans un réseau traditionnel.

Chapitre 6

Conclusion et Perspectives

6.1 Bilan du Projet

Ce projet a permis de concevoir et déployer une architecture SDN complète répondant aux exigences de gestion dynamique du trafic et de sécurité réseau. Les six phases du projet ont été menées à bien :

- **Phase 1** : Étude bibliographique et état de l’art sur le SDN, OpenFlow et les contrôleurs.
- **Phase 2** : Déploiement de l’environnement Mininet + Ryu avec topologie arbre et Fat-Tree.
- **Phase 3** : Développement des modules de gestion du trafic (monitoring, routage, QoS, load balancing).
- **Phase 4** : Développement du module de sécurité (détection DDoS, mitigation, pare-feu stateful).
- **Phase 5** : Tests et évaluation sur 4 scénarios avec comparaison SDN vs réseau traditionnel.
- **Phase 6** : Rédaction du rapport et préparation de la soutenance.

6.2 Résultats Obtenus

Les principaux résultats obtenus sont :

TABLE 6.1 – Récapitulatif des résultats obtenus

Module	Métrique	Résultat
Trafic normal	Débit TCP	23,5 Gbps
Trafic normal	Latence avg	0,134 ms
Trafic normal	Jitter UDP	0,001 ms
Trafic normal	Perte paquets	0%
Sécurité	Détection DDoS	5 secondes
Sécurité	Mitigation	< 1 seconde
QoS	Débit TCP prioritaire	19,7 Gbps
QoS	Latence ICMP	0,219 ms
Comparaison	Réduction latence	52% vs traditionnel
Overhead	CPU contrôleur	21,5%
Overhead	Mémoire	65 MB

6.3 Contributions

Les principales contributions de ce projet sont :

1. Un **module de détection DDoS** par entropie de Shannon avec trois algorithmes complémentaires : taux de paquets/sec, entropie source/destination et détection de scan de ports.
2. Un **système de mitigation automatique graduée** en trois niveaux progressifs : rate limiting (priorité 80), redirection honeypot (priorité 90) et blocage DROP (priorité 100), permettant une réponse proportionnelle à la sévérité de l'attaque.
3. Un **pare-feu stateful SDN** avec suivi des états TCP (SYN, ESTABLISHED, CLOSING) et règles ACL configurables, entièrement implémenté au niveau du contrôleur Ryu.
4. Un **module QoS dynamique** à trois niveaux de priorité (HAUTE, MOYENNE, BASSE) installé via des règles OpenFlow directement sur les switches.
5. Une **évaluation comparative complète** sur quatre scénarios (trafic normal, DDoS, panne de lien, QoS) montrant la supériorité du SDN sur le réseau traditionnel en termes de latence, sécurité et gestion du trafic.

6.4 Difficultés Rencontrées

Plusieurs difficultés ont été surmontées durant ce projet :

- **Compatibilité Python 3.12/Ryu** : Ryu étant conçu pour Python 2/3.6, des avertissements de dépréciation d'Eventlet ont nécessité l'utilisation d'un environnement virtuel dédié.
- **Calibrage des seuils DDoS** : Les seuils de détection (2 pkt/s) ont dû être ajustés pour éviter les faux positifs dans l'environnement simulé.
- **Règles OpenFlow conflictuelles** : La gestion des priorités entre les règles de sécurité (priorité 100) et de routage (priorité 1) a nécessité une conception soigneuse.
- **Idle timeout** : Le paramètre `idle=2s` dans `ddos_detector.py` était nécessaire pour forcer la remontée des paquets au contrôleur et maintenir les compteurs DDoS à jour.

6.5 Perspectives

Plusieurs pistes d'amélioration peuvent être envisagées :

6.5.1 Court Terme

- **Interface web de supervision** : Développement d'un tableau de bord temps réel avec Flask/Django pour visualiser les métriques réseau, les alertes DDoS et les règles de flux actives.
- **Amélioration du reroutage** : Implémenter un mécanisme de reroutage automatique lors d'une panne de lien, exploitant les chemins alternatifs disponibles.
- **Tests sur Fat-Tree** : Évaluation des performances sur la topologie Fat-Tree $k=8$ pour un environnement datacenter plus réaliste.

6.5.2 Moyen Terme

- **Machine Learning** : Intégration d'algorithmes d'apprentissage automatique (SVM, Random Forest) pour améliorer la précision de la détection DDoS et réduire les faux positifs.
- **Scalabilité** : Étude du comportement du contrôleur Ryu avec un nombre croissant de switches (50, 100, 500) et évaluation des limites de passage à l'échelle.
- **Multi-contrôleurs** : Déploiement d'une architecture SDN distribuée avec plusieurs contrôleurs Ryu pour la haute disponibilité.

6.5.3 Long Terme

- **Déploiement en production** : Migration vers le contrôleur ONOS pour un déploiement en environnement de production avec haute disponibilité.
- **SDN et IoT** : Extension de l'architecture pour la gestion du trafic IoT avec des politiques de sécurité adaptées aux contraintes des objets connectés.
- **P4 Programming** : Exploration du langage P4 pour programmer le plan de données des switches et implémenter la détection DDoS directement dans le datapath.

6.6 Conclusion Générale

Ce projet a démontré la faisabilité et l'efficacité d'une architecture SDN pour la gestion dynamique du trafic et la sécurité réseau. Le contrôleur Ryu, combiné à l'émulateur Mininet, offre un environnement de développement et de test idéal pour prototyper et valider des solutions réseau innovantes.

Les résultats obtenus confirment les avantages du SDN : une latence réduite de 52% par rapport au réseau traditionnel, une détection DDoS automatique en 5 secondes avec mitigation en moins d'une seconde, et une gestion QoS dynamique permettant de garantir la qualité de service pour les applications critiques.

Le code source complet, documenté et commenté est disponible sur GitHub : <https://github.com/thioraminata753/projet-sdn>.

Bibliographie

- [1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, J. Turner, *OpenFlow : Enabling Innovation in Campus Networks*, ACM SIGCOMM Computer Communication Review, vol. 38, no. 2, pp. 69–74, 2008.
- [2] D. Kreutz, F. Ramos, P. Verissimo, C. Rothenberg, S. Azodolmolky, S. Uhlig, *Software-Defined Networking : A Comprehensive Survey*, Proceedings of the IEEE, vol. 103, no. 1, pp. 14–76, 2015.
- [3] S. Sezer, S. Scott-Hayward, P. Chouhan, B. Fraser, D. Lake, J. Finnegan, N. Viljoen, M. Miller, N. Rao, *Are We Ready for SDN ? Implementation Challenges for Software-Defined Networks*, IEEE Communications Magazine, vol. 51, no. 7, pp. 36–43, 2013.
- [4] S. Shin, V. Yegneswaran, P. Porras, G. Gu, *AVANT-GUARD : Scalable and Vigilant Switch Flow Management in Software-Defined Networks*, ACM CCS, pp. 413–424, 2013.
- [5] R. Braga, E. Mota, A. Passito, *Lightweight DDoS Flooding Attack Detection Using NOX/OpenFlow*, IEEE Local Computer Network Conference, pp. 408–415, 2010.
- [6] S. Mousavi, M. St-Hilaire, *Early Detection of DDoS Attacks against SDN Controllers*, IEEE International Conference on Computing, Networking and Communications, pp. 77–81, 2015.
- [7] M. Al-Fares, A. Loukissas, A. Vahdat, *A Scalable, Commodity Data Center Network Architecture*, ACM SIGCOMM Computer Communication Review, vol. 38, no. 4, pp. 63–74, 2008.
- [8] Ryu SDN Framework, *Documentation officielle Ryu*, <https://ryu-sdn.org/>, 2024.
- [9] Mininet Team, *Documentation officielle Mininet*, <http://mininet.org/>, 2024.
- [10] Open Networking Foundation, *OpenFlow Switch Specification Version 1.3*, <https://opennetworking.org/>, 2012.

Annexe A

Guide d'Installation et de Déploiement

A.1 Prérequis

- Ubuntu 22.04 LTS (ou Debian 11+)
- Python 3.12+
- Mininet 2.3+
- Open vSwitch 2.13+
- Git
- iperf3, hping3, tshark, Wireshark

A.2 Installation Pas à Pas

Listing A.1 – Installation complète de l'environnement

```
# Etape 1 : Cloner le depot GitHub
git clone https://github.com/thioraminata753/projet-sdn
cd projet-sdn

# Etape 2 : Creer l'environnement virtuel Python
python3 -m venv venv
source venv/bin/activate

# Etape 3 : Installer les dependances
pip install ryu matplotlib numpy
```

```
# Etape 4 : Verifier l'installation
ryu-manager --version
mn --version
iperf3 --version
```

A.3 Lancement des Modules

Listing A.2 – Lancement du projet SDN

```
# Terminal 1 : Activer venv et lancer Ryu
source ~/projet-sdn/venv/bin/activate

# Module gestion trafic (default)
ryu-manager traffic_manager.py

# Module securite unifie
ryu-manager security_manager.py

# Module detection DDoS
ryu-manager ddos_detector.py

# Module QoS
ryu-manager qos.py

# Terminal 2 : Lancer Mininet
sudo mn --controller=remote,ip=127.0.0.1,port=6633 \
        --switch=ovsk,protocols=OpenFlow13 \
        --topo=tree,depth=2,fanout=4 --mac

# Terminal 3 : Verifier les logs
tail -f /tmp/sdn_logs/security_manager.log
```

A.4 Tests de Validation

Listing A.3 – Commandes de test dans Mininet

```
# Test connectivite de base
```

```
mininet> pingall
```

```
# Test latence (100 paquets)
```

```
mininet> h1 ping -c 100 10.0.0.16
```

```
# Test debit TCP
```

```
mininet> h1 iperf3 -s &
```

```
mininet> h16 iperf3 -c 10.0.0.1 -t 10
```

```
# Test debit UDP + jitter
```

```
mininet> h16 iperf3 -c 10.0.0.1 -u -b 1G -t 10
```

```
# Simulation attaque DDoS
```

```
mininet> h1 hping3 -S -p 80 --flood 10.0.0.16 &
```

```
# Simulation panne lien
```

```
mininet> link s1 s2 down
```

```
mininet> link s1 s2 up
```

```
# Generation graphiques
```

```
python3 graphiques_phase5.py
```


Annexe B

Structure du Dépôt GitHub

Le code source complet est disponible à l'adresse : <https://github.com/thioraminata753/projet-sdn>

TABLE B.1 – Structure du dépôt GitHub

Fichier/Dossier	Description
traffic_manager.py	Module principal gestion trafic
ddos_detector.py	Détection DDoS par entropie
firewall.py	Pare-feu stateful ACL
security_manager.py	Module sécurité unifié
qos.py	Gestion QoS 3 niveaux
monitoring.py	Surveillance trafic
routing.py	Routage dynamique
load_balancer.py	Load balancing Round-Robin
mitigation.py	Mitigation automatique
fat_tree.py	Topologie Fat-Tree k=4
graphiques_phase5.py	Génération graphiques
tests/	Scripts tests automatisés
captures/	Captures d'écran des tests
resultats_graphiques/	Graphiques matplotlib
rapports/	Rapport intermédiaire PDF

Annexe C

Paramètres de Configuration

TABLE C.1 – Paramètres configurables des modules

Paramètre	Valeur	Module
SEUIL_PAQUETS_PAR_SEC	2	ddos_detector.py
SEUIL_ENTROPIE	0.5	ddos_detector.py
SEUIL_FLUX_PAR_IP	10	ddos_detector.py
INTERVALLE_ANALYSE	5s	ddos_detector.py
HONEYPOT_IP	10.0.0.16	security_manager.py
DROP_TIMEOUT	60s	ddos_detector.py
HONEYPOT_TIMEOUT	120s	mitigation.py
RATE_LIMIT_TIMEOUT	30s	mitigation.py
idle_timeout (routage)	30s	traffic_manager.py
idle_timeout (DDoS)	2s	ddos_detector.py
Port contrôleur	6633	Tous les modules